CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Section 4: Structured Outputs, Extraction & Automated Review Design

Exam objectives covered

1. Design human review routing strategies that direct extractions to reviewers based on confidence scores, document characteristics, and field-level ambiguity rather than random sampling.
2. Select the appropriate API processing mode—synchronous Messages API or asynchronous Message Batches API—based on latency requirements, workflow blocking behavior, and acceptable processing windows.
3. Design specialized review passes that separate concerns—security, business logic, API design—into focused prompts with dedicated few-shot examples, preventing recall trade-offs from competing concerns in a single prompt.
4. Select and implement the most reliable structured output method—tool use with JSON schema, prompt-based formatting, or prefilled responses—based on required schema compliance strictness.
5. Design extraction schemas with optional fields, nullable values, and appropriate enum definitions that allow the model to accurately represent missing or ambiguous information without fabricating values.
6. Implement tool use with defined JSON schemas to enforce structured output compliance, and configure `tool_choice` to guarantee tool invocation when conversational responses would cause downstream failures.
7. Resolve structured output truncation failures by splitting large review tasks into smaller scoped API calls and merging resulting data structures, rather than increasing max_tokens beyond practical limits.
8. Apply extraction accuracy patterns—structured schemas with optional fields, format normalization instructions, and few-shot examples—to reduce hallucination and improve consistency across varied document formats.
9. Reduce automated review false positive rates by supplying project-specific conventions, accepted patterns, and exclusion criteria as persistent context applied on every review.
10. Design prompt criteria that define explicit inclusion and exclusion boundaries, preventing the model from generating findings or extractions in categories where performance is unreliable.
11. Configure the `tool_choice` parameter to guarantee tool invocation when structured output is required, and sequence multi-tool workflows so prerequisite data is obtained before dependent tools are called.

Q1.

A team reviews invoice extractions by sending 10% of documents to humans at random. Error analysis shows most mistakes occur on low-confidence totals, handwritten scans, and missing vendor IDs. Which redesign best aligns with robust human review routing?

A. Keep random sampling, but increase the sample to 25% B. Route documents to review based on confidence thresholds, scan quality, and field-level ambiguity C. Review only the largest invoices because they have the highest business impact D. Send every extraction to the same reviewer to improve consistency

Answer: B **Explanation:** Confidence-based routing with document characteristics and field-level ambiguity directly targets where the model is least reliable. A wastes reviewer time on easy cases, C ignores many real failure modes, and D does not improve selection quality—only reviewer assignment consistency.

Q2.

You need to process 500,000 archived contracts overnight, and no user is waiting on the result interactively. Which API mode is most appropriate?

A. Synchronous Messages API, because it returns immediate responses B. Asynchronous Message Batches API, because throughput matters more than per-request latency C. Synchronous Messages API, because batching increases hallucinations D. Either API is equally ideal because latency never matters for extraction

Answer: B **Explanation:** The batch workflow is designed for large-scale asynchronous processing where blocking an interactive workflow is unnecessary. A is less operationally efficient at this scale, C is incorrect because batching does not inherently increase hallucinations, and D ignores the important distinction between interactive and non-interactive workloads.

Q3.

An application must extract a shipping address and immediately display it to a call-center agent during a live customer interaction. Which API mode is best?

A. Message Batches API, because it is better for all extraction use cases B. Messages API, because low latency and immediate response are required C. Either one, because structured output guarantees identical timing D. Message Batches API, because the task is small

Answer: B **Explanation:** When a workflow is user-blocking and requires immediate output, the synchronous Messages API is the appropriate choice. A and D ignore the latency requirement, while C wrongly assumes output format guarantees say anything about response timing.

Q4.

A single automated review prompt asks the model to find security vulnerabilities, business logic bugs, API design flaws, performance issues, and style inconsistencies in one pass. Results are noisy and incomplete. What is the best redesign?

A. Increase temperature so the model explores more possibilities B. Split the review into specialized passes with focused prompts and examples for each concern C. Ask the model to "think harder" in the same prompt D. Remove examples so the model has more room for reasoning

Answer: B **Explanation:** Separate passes reduce recall trade-offs caused by competing objectives in one prompt and allow targeted instructions and few-shot examples per concern. A generally increases variance, C is vague and does not fix prompt competition, and D removes helpful guidance that often improves review quality.

Q5.

Which structured output approach is most reliable when downstream systems require strict schema compliance?

A. Ask the model to "return valid JSON" in plain text B. Use prefilled response text that starts with `{` and hope completion follows the format C. Use tool invocation with a defined JSON schema D. Let the model answer conversationally and parse it with regex later

Answer: C **Explanation:** Tool use with a JSON schema provides the strongest enforcement mechanism for structured outputs and is the most reliable for downstream automation. A and B can still drift or truncate improperly, and D is brittle and failure-prone.

Q6.

You are designing a schema for extracting "middle_name" from customer intake forms, but many forms omit it. What schema design is best?

A. Make `middle_name` required and instruct the model to infer it when possible B. Make `middle_name` optional or nullable so absence can be represented explicitly C. Use an empty string only; nulls should never appear in extraction outputs D. Replace `middle_name` with a boolean field called `has_middle_name` only

Answer: B **Explanation:** Optional or nullable fields let the model represent genuinely missing data without fabricating values. A encourages guessing, C can blur the distinction between absent and intentionally blank values, and D loses the actual extracted value when present.

Q7.

An extraction pipeline fails whenever the model replies conversationally instead of returning the tool payload. What configuration most directly addresses this?

A. Increase `max_tokens` B. Set `tool_choice` to guarantee the required tool is invoked C. Lower temperature to 0 D. Add "please be careful" to the prompt

Answer: B **Explanation:** For workflows that require structured output, `tool_choice` should be configured to force tool invocation and prevent free-form conversational responses. A affects length, not tool use; C may improve consistency but does not guarantee tool calling; D is too weak to enforce protocol compliance.

Q8.

A code review task produces truncated structured output because the repository diff is too large and the resulting findings exceed practical token limits. What is the best fix?

A. Keep increasing `max_tokens` until the full response fits B. Ask for fewer details but keep the whole review in one call C. Split the review into smaller scoped calls and merge the results D. Switch to free-form text instead of structured output

Answer: C **Explanation:** Large review tasks should be decomposed into smaller units to avoid truncation while preserving structured outputs that can later be merged. A is not practical beyond a point, B may still miss important content, and D sacrifices reliability and downstream usability.

Q9.

Which schema most reduces hallucination when extracting a document's payment status?

A. A free-text `status_description` field with no constraints B. An enum such as `paid`, `unpaid`, `pending`, plus nullable support if unclear C. A required boolean `is_paid` even when the document is ambiguous D. No schema—just ask the model to summarize the payment state

Answer: B **Explanation:** A constrained enum helps normalize outputs and reduces invented variants, while nullability lets the model express uncertainty honestly. A invites inconsistent phrasing, C forces false precision, and D removes structure entirely.

Q10.

A review system flags internal helper functions for "missing public API documentation," even though the project explicitly excludes internal-only code from that rule. What is the best way to reduce these false positives?

A. Tell the model to be stricter about documentation B. Provide project-specific conventions and exclusion criteria in every review prompt C. Increase the sample size of reviewed files D. Remove all examples to avoid biasing the reviewer

Answer: B **Explanation:** Persistent project-specific context—such as accepted patterns and exclusions—is essential for lowering false positive rates in automated review. A may worsen noise, C affects coverage rather than rule accuracy, and D removes useful calibration.

Q11.

Which prompt instruction best defines reliable inclusion and exclusion boundaries for an automated review?

A. "Find anything suspicious or potentially bad." B. "Report every possible issue you can imagine." C. "Report only SQL injection risks caused by unsanitized user input reaching query construction; ignore style, performance, and authorization issues." D. "Review broadly, but prefer important issues."

Answer: C **Explanation:** Explicit inclusion and exclusion boundaries constrain the model to categories where performance is intended and measurable. A, B, and D are too vague and

encourage drift into unreliable or unwanted issue classes.

Q12.

A workflow requires first extracting a purchase order ID, then calling a lookup tool that fetches associated records using that ID. Which orchestration design is best?

A. Let the model call both tools in any order it chooses B. Force the lookup tool first so the model has more context C. Sequence the workflow so extraction happens first, then the dependent lookup tool is called D. Merge both tools into one giant tool with no required parameters

Answer: C **Explanation:** Dependent multi-tool workflows should respect prerequisites so downstream tools receive the data they require. A risks invalid ordering, B is impossible without the ID, and D reduces clarity and often makes validation harder.

Q13.

A team currently instructs the model: "Output JSON exactly in this format…" and usually gets good results. However, occasional formatting drift breaks production parsing. What is the best improvement?

A. Add more exclamation marks to emphasize the instruction B. Switch to tool use with schema validation for stronger compliance guarantees C. Lower temperature and keep the same prompt-only approach forever D. Parse the output with more permissive regexes

Answer: B **Explanation:** Prompt-based formatting can work, but if occasional drift causes downstream failures, a schema-enforced tool approach is more reliable. A is ineffective, C may reduce but not eliminate failures, and D increases brittleness rather than fixing the root cause.

Q14.

A field called `termination_date` appears only in some contracts, and sometimes the contract explicitly states that there is no termination date. Which representation is best?

A. Always require a date string and use today's date if missing B. Omit the field in all cases to avoid mistakes C. Allow the field to be nullable or optional so the output can distinguish present versus absent/none D. Convert the field to an enum of `short`, `medium`, or `long`

Answer: C **Explanation:** Optionality or nullability lets the extraction represent both "field not present" and "no date applies" without inventing a value. A fabricates data, B discards useful information, and D changes the meaning of the field entirely.

Q15.

Which scenario most strongly favors prompt-based formatting or prefilled responses over tool use?

A. A compliance pipeline where invalid schema output causes the transaction to fail B. A production data ingestion system that requires strict field typing C. A lightweight one-off formatting task where occasional manual correction is acceptable D. A multi-stage workflow that depends on guaranteed machine-readable output

Answer: C **Explanation:** Prompt-based or prefilled formatting is more suitable when stakes are lower and strict schema enforcement is not critical. A, B, and D all describe cases where tool-based structured output is the safer and more reliable option.

Q16.

An extraction prompt says, "If the tax ID is missing, provide your best guess from context." What is the main problem with this instruction?

A. It will slow down the response unnecessarily B. It encourages fabrication instead of representing uncertainty explicitly C. It makes enum definitions impossible D. It prevents batch processing

Answer: B **Explanation:** Extraction systems should allow the model to express missing or ambiguous information without guessing. A is not the key issue, C is unrelated, and D has nothing to do with whether guessing is encouraged.

Q17.

A team wants to improve extraction consistency across diverse vendor forms. Which combination is most effective?

A. Higher temperature and shorter prompts B. Structured schema, normalization instructions, and few-shot examples covering variation C. Free-text summaries followed by manual cleanup D. Randomly rotating prompts to avoid overfitting

Answer: B **Explanation:** Combining a schema with normalization guidance and representative few-shot examples is a core accuracy pattern for varied document formats. A typically increases inconsistency, C reduces automation quality, and D introduces unnecessary variability.

Q18.

You are reviewing a legacy codebase and need separate outputs for security findings, business logic defects, and API design issues. Why is three focused review passes usually better than one combined pass?

A. Because the model cannot reason about more than one topic at a time B. Because separate passes reduce competing recall demands and allow tailored instructions/examples per category C. Because combined prompts always exceed token limits D. Because only security reviews can use few-shot examples

Answer: B **Explanation:** Focused passes improve precision and recall by isolating concerns and supplying category-specific guidance, reducing trade-offs within a single overloaded prompt. A is too absolute, C is not always true, and D is false because few-shot examples help many review categories.

Q19.

A downstream service crashes if the model ever returns natural language instead of a JSON tool payload. Which setup is most appropriate?

A. Use tool schema plus `tool_choice` forcing the required tool B. Use prompt-based JSON with temperature 0.1 C. Use a prefilled `{` character and hope completion stays valid D. Accept conversational output and transform it later

Answer: A **Explanation:** When conversational deviation is unacceptable, both a defined tool schema and forced tool invocation are appropriate. B and C can still fail occasionally, and D moves critical risk downstream.

Q20.

Which review-routing policy best reflects mature handling of extraction risk?

A. Review every twentieth document regardless of content B. Review only documents from new customers C. Route by confidence, document complexity, ambiguous fields, and business-critical field failures D. Skip human review if the model has performed well for the last week

Answer: C **Explanation:** Mature review routing prioritizes signals that correlate with likely extraction error and business impact. A is arbitrary, B is too narrow, and D ignores that performance varies by document type, field, and ambiguity level.